



Java ArrayList

[< Previous](#)[Next >](#)

Java ArrayList

An **ArrayList** is like a resizable array.

It is part of the **java.util** package and implements the List interface.

The difference between a built-in array and an **ArrayList** in Java, is that the size of an array cannot be modified (if you want to add or remove elements to/from an array, you have to create a new one). While elements can be added and removed from an **ArrayList** whenever you want.

Create an ArrayList

To use an **ArrayList**, you must first import it from **java.util**:

Example

[Get your own Java Server](#)

Create an **ArrayList** object called **cars** that will store strings:

```
import java.util.ArrayList; // Import the ArrayList class
```



Now you can use methods like `add()`, `get()`, `set()`, and `remove()` to manage your list of elements.

Add Elements

To add elements to an `ArrayList`, use the `add()` method:

Example

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> cars = new ArrayList<String>();
        cars.add("Volvo");
        cars.add("BMW");
        cars.add("Ford");
        cars.add("Mazda");
        System.out.println(cars);
    }
}
```

Try it Yourself »

You can also add an element at a specified position by referring to the index number:

Example

```
import java.util.ArrayList;
```



```
cars.add("BMW");  
cars.add("Ford");  
  
cars.add(0, "Mazda"); // Insert element at the beginning of the list (0  
  
System.out.println(cars);  
}  
}
```

Try it Yourself »



An **ArrayList** keeps elements in the same order you add them, so the first item you add will be at index 0, the next at index 1, and so on.

Access an Element

To access an element in the **ArrayList**, use the **get()** method and refer to the index number:

Example

```
cars.get(0); // Get the first element
```

Try it Yourself »

[REMOVE ADS](#)

Change an Element

To modify an element, use the `set()` method and refer to the index number:

Example

```
cars.set(0, "Opel");
```

[Try it Yourself »](#)

Remove an Element

To remove an element, use the `remove()` method and refer to the index number:

Example

```
cars.remove(0);
```



To remove all the elements in the `ArrayList`, use the `clear()` method:

Example

```
cars.clear();
```

[Try it Yourself »](#)

ArrayList Size

To find out how many elements an `ArrayList` have, use the `size` method:

Example

```
cars.size();
```

[Try it Yourself »](#)

Loop Through an ArrayList

Loop through the elements of an `ArrayList` with a `for` loop, and use the `size()` method to specify how many times the loop should run:

Example



Tutorials ▼

References ▼

Exercises ▼



Sign In



CSS

JAVASCRIPT

SQL

PYTHON

JAVA

PHP

HOW TO

W3.CSS

C

```
cars.add("Volvo");
cars.add("BMW");
cars.add("Ford");
cars.add("Mazda");
for (int i = 0; i < cars.size(); i++) {
    System.out.println(cars.get(i));
}
}
```

[Try it Yourself »](#)

You can also loop through an `ArrayList` with the **for-each** loop:

Example

```
public class Main {
    public static void main(String[] args) {
        ArrayList<String> cars = new ArrayList<String>();
        cars.add("Volvo");
        cars.add("BMW");
        cars.add("Ford");
        cars.add("Mazda");
        for (String i : cars) {
            System.out.println(i);
        }
    }
}
```

[Try it Yourself »](#)



(objects) of type "String". Remember that a **String** in Java is an object (not a primitive type). To use other types, such as **int**, you must specify an equivalent wrapper class: **Integer**. For other primitive types, use: **Boolean** for boolean, **Character** for char, **Double** for double, etc:

Example

Create an **ArrayList** to store numbers (add elements of type **Integer**):

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> myNumbers = new ArrayList<Integer>();
        myNumbers.add(10);
        myNumbers.add(15);
        myNumbers.add(20);
        myNumbers.add(25);
        for (int i : myNumbers) {
            System.out.println(i);
        }
    }
}
```

Try it Yourself »

Sort an ArrayList

Another useful class in the **java.util** package is the **Collections** class, which include the **sort()** method for sorting lists alphabetically or numerically:



```
import java.util.ArrayList;
import java.util.Collections; // Import the Collections class

public class Main {
    public static void main(String[] args) {
        ArrayList<String> cars = new ArrayList<String>();
        cars.add("Volvo");
        cars.add("BMW");
        cars.add("Ford");
        cars.add("Mazda");
        Collections.sort(cars); // Sort cars
        for (String i : cars) {
            System.out.println(i);
        }
    }
}
```

Try it Yourself »

Example

Sort an ArrayList of Integers:

```
import java.util.ArrayList;
import java.util.Collections; // Import the Collections class

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> myNumbers = new ArrayList<Integer>();
        myNumbers.add(33);
        myNumbers.add(15);
        myNumbers.add(20);
    }
}
```

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Sign In](#)[CSS](#)[JAVASCRIPT](#)[SQL](#)[PYTHON](#)[JAVA](#)[PHP](#)[HOW TO](#)[W3.CSS](#)[C](#)

```
Collections.sort(myNumbers); // Sort myNumbers
```

```
for (int i : myNumbers) {  
    System.out.println(i);  
}  
}  
}
```

[Try it Yourself »](#)

The var Keyword

From Java 10, you can use the **var** keyword to declare an **ArrayList** variable without writing the type twice. The compiler figures out the type from the value you assign.

This makes code shorter, **but many developers still use the full type for clarity**. Since **var** is valid Java, you may see it in other code, so it's good to know that it exists:

Example

```
// Without var  
ArrayList<String> cars = new ArrayList<String>();  
  
// With var  
var cars = new ArrayList<String>();
```

[Try it Yourself »](#)



```
import java.util.List;
import java.util.ArrayList;

List<String> cars = new ArrayList<>();
```

Try it Yourself »

This means the variable (cars) is declared as a [List](#) (the interface), but it stores an [ArrayList](#) object (the actual list). Since [ArrayList](#) implements the [List](#) interface, this is possible.

It works the same way, but some developers prefer this style because it gives them more flexibility to change the type later.

Complete ArrayList Reference

For a complete reference of ArrayList methods, go to our [Java ArrayList Reference](#).

Exercise [?]

True or False:

An ArrayList is a fixed-size array, while a regular array is resizable.

☐ True

☐ False

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Sign In](#)[CSS](#)[JAVASCRIPT](#)[SQL](#)[PYTHON](#)[JAVA](#)[PHP](#)[HOW TO](#)[W3.CSS](#)[C](#)[< Previous](#)[Sign in to track progress](#)[Next >](#)

ADVERTISEMENT





Tutorials ▼

References ▼

Exercises ▼



Sign In

- ☰
- .
- CSS
- JAVASCRIPT
- SQL
- PYTHON
- JAVA
- PHP
- HOW TO
- W3.CSS
- C

Document your skills with
W3Schools Certificates

~~\$1,995~~ **\$699**

Save 65% 🏷️

COLOR PICKER



ADVERTISEMENT



REMOVE ADS

ADVERTISEMENT

ADVERTISEMENT



Tutorials ▼

References ▼

Exercises ▼



Sign In

- ☰
- CSS
- JAVASCRIPT
- SQL
- PYTHON
- JAVA
- PHP
- HOW TO
- W3.CSS
- C



PLUS

SPACES

GET CERTIFIED

FOR TEACHERS

FOR BUSINESS

CONTACT US

Top Tutorials

- HTML Tutorial
- CSS Tutorial
- JavaScript Tutorial
- How To Tutorial
- SQL Tutorial
- Python Tutorial
- W3.CSS Tutorial
- Bootstrap Tutorial
- PHP Tutorial
- Java Tutorial
- C++ Tutorial
- jQuery Tutorial

Top References

- HTML Reference
- CSS Reference
- JavaScript Reference
- SQL Reference



AngularJS Reference

jQuery Reference

Top Examples

- HTML Examples
- CSS Examples
- JavaScript Examples
- How To Examples
- SQL Examples
- Python Examples
- W3.CSS Examples
- Bootstrap Examples
- PHP Examples
- Java Examples
- XML Examples
- jQuery Examples

Get Certified

- HTML Certificate
- CSS Certificate
- JavaScript Certificate
- Front End Certificate
- SQL Certificate
- Python Certificate
- PHP Certificate
- jQuery Certificate
- Java Certificate
- C++ Certificate
- C# Certificate
- XML Certificate



FORUM

ABOUT

ACADEMY

W3Schools is optimized for learning and training. Examples might be simplified to improve reading and learning.

Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness of all content. While using W3Schools, you agree to have read and accepted our [terms of use](#), [cookie and privacy policy](#).

[Copyright 1999-2025](#) by Refsnes Data. All Rights Reserved. [W3Schools is Powered by W3.CSS](#).